



UNIVERSITY OF KELANIYA
SRI LANKA
RESEARCH PROPOSAL

Machine Learning-Based SQL Query Performance Prediction

Using Multi-Modal Features (Query Plan, Semantic, and System Metrics)

IN PARTIAL FULFILLMENT FOR THE DEGREE OF
BACHELOR OF SCIENCE HONOURS IN SOFTWARE ENGINEERING
FACULTY OF SCIENCE
UNIVERSITY OF KELANIYA

SUBMITTED BY:

Kirisigan Kannathasan | Nirojan Kunesan | Thivya Mahendran | Jashvanth Balakrishnan
Academic Year 2024 / 2025

1. Group Member Details and Assigned Roles

The following table presents the group composition and individual role assignments for this research project.

Student No.	Name	Role	Responsibilities
SE/2021/007	Kirisigan Kannathan	Project Manager	Task allocation, project planning, progress monitoring, team coordination, and overall project supervision.
SE/2021/014	Nirojan Kunesan	Documentation Lead	Preparing reports, maintaining documentation, formatting, and managing references.
SE/2021/022	Thivya Mahendran	Research Lead	Conducting the literature review, identifying research gaps, and supporting methodology.
SE/2021/055	Jashvanth Balakrishnan	Lead Developer	System development, data processing, model implementation, and testing.

2. Introduction

This research proposal focuses on improving SQL query performance prediction using machine learning techniques. It aims to address limitations in traditional cost-based query optimizers by integrating multiple feature types to enhance prediction accuracy.

2.1 Background

Database management systems (DBMS) underpin virtually every modern enterprise application, from e-commerce platforms to financial services. As organisations accumulate larger and more complex datasets, the performance of SQL queries has become a critical operational concern. Database query optimizers, such as those in PostgreSQL, generate query execution plans and estimate execution cost using static, rule-based cost models. These models account for factors such as CPU cost, disk I/O, and cardinality estimates; however, in practice they frequently diverge from actual execution times, particularly under complex multi-join queries, skewed data distributions, and dynamic concurrent workloads.

2.2 Importance

Accurate prediction of SQL query execution time prior to execution has significant practical value. It enables database administrators to perform proactive resource allocation, optimize workload scheduling, and guide query plan selection, directly reducing latency and improving throughput. For organisations operating at scale, even marginal improvements in prediction accuracy translate into measurable infrastructure cost savings. Beyond its industry impact, this research contributes to the

growing field of machine learning applied to database internals, an area of increasing academic and commercial interest.

2.3 Motivation

Existing approaches to query performance prediction typically rely on a single feature modality, either query execution plan features, SQL textual semantics, or runtime system metrics, in isolation. No reviewed study integrates all three simultaneously (Pathak & Mankodi, 2025). This leaves a clear gap in both prediction accuracy and model generalizability. This research is motivated by the hypothesis that a unified multi-modal model, trained on large real-world benchmark data, will outperform single-modality baselines and traditional cost-based estimates, providing a more robust and practically deployable prediction solution.

3. Problem Statement

Traditional DBMS cost-based query optimizers rely on static models and cardinality estimation techniques that frequently fail to capture real-world query behavior, particularly in complex multi-join queries, skewed data distributions, and dynamic workloads (Lan et al., 2021; Wu et al., 2013). These models also omit runtime system conditions such as CPU utilization, memory usage, and disk I/O, leading to inaccurate execution time predictions and suboptimal query plan selection (Wu et al., 2013). This limitation directly increases query latency and causes inefficient resource utilization in production environments.

Primary Research Question: Can a machine learning model that integrates multi-modal features, namely query execution plan features, semantic SQL features, and runtime system metrics, achieve lower prediction error compared to traditional PostgreSQL cost-based execution time estimates?

Secondary research questions that guide this investigation:

- Which feature modality (query plan, semantic, or system metrics) contributes most significantly to prediction accuracy?
- To what extent does the inclusion of runtime system metrics improve prediction performance over models using only plan and semantic features?
- How well does the proposed multi-modal model generalize across different query templates and workload types (TPC-H vs TPC-DS)?
- How does the performance of XGBoost compare to Random Forest and baseline linear models in this multi-modal setting?

4. Research Objectives

The following objectives are specific, measurable, and achievable within the project timeline:

- **Pipeline development:** Develop a data collection pipeline using PostgreSQL and EXPLAIN ANALYZE to systematically gather execution plans, query statistics, and runtime metrics from TPC-H and TPC-DS benchmark workloads (minimum 5000 queries), addressing the small dataset limitation identified in (Pathak & Mankodi, 2025).

- **Feature engineering:** Extract and engineer multi-modal features spanning three dimensions: (a) query plan features, including operator types, cardinality, cost estimates, and tree structure; (b) semantic features, including join count, predicate complexity, subquery nesting; and (c) system-level metrics, including CPU, memory, and disk I/O utilisation.
- **Model development:** Build, train, and optimise two machine learning regression models, Random Forest Regressor and XGBoost Regressor on the collected multi-modal feature set to predict SQL query execution time, building on the demonstrated superiority of XGBoost for this task (Pathak & Mankodi, 2025; Li et al., 2012).
- **Model evaluation:** Evaluate model performance using MAE, MSE, and R^2 Score, and demonstrate statistically significant improvement over both traditional PostgreSQL cost estimates and the single-modality baseline established by Pathak and Mankodi (2025).
- **Deployable framework:** Produce a generalizable, documented machine learning framework suitable for deployment in real PostgreSQL database environments, with a Python API for pre-execution query time prediction.

5. Preliminary Literature Review

SQL query performance prediction is challenging due to complex workloads and the limitations of static cost models. Recent machine learning and hybrid approaches aim to improve accuracy. This review summarizes key contributions and identifies the specific gap this study addresses.

5.1 Traditional and Adaptive Cost-Based Estimation

Conventional DBMS optimizers such as PostgreSQL and MySQL estimate execution time using static cost models based on CPU, I/O, and cardinality estimates. Lan et al. (2021) identified cardinality estimation as the primary source of prediction error, noting that inaccuracies compound significantly in complex queries. Wu et al. (2013) demonstrated that traditional models fail under concurrent workloads and proposed a queueing-based analytic model incorporating system-level factors, achieving improved accuracy. Vasilenko et al. (2024) introduced an adaptive cost model that dynamically updates parameters using runtime statistics; however, this approach does not address cardinality estimation errors directly.

5.2 Machine Learning with Shallow Feature Sets

Kleerekoper et al. (2019) showed that using only the optimizer cost as a feature, a nearest-neighbour model could achieve very low prediction error, highlighting the predictive value of cost estimates. Li et al. (2012) contributed in two key areas: improving selectivity estimation using sampling techniques, and applying boosted regression trees (MART) for resource consumption prediction, enhancing modeling of non-linear resource usage relationships.

5.3 Query Plan Feature-Based Deep Learning

Marcus and Papaemmanouil (2019) proposed Plan-Structured Neural Networks (PSNN), which model query execution plans as tree structures within neural networks, improving prediction accuracy at high computational cost. Pathak and Mankodi (2025) combined plan features, structural features, and semantic embeddings, with their XGBoost model outperforming other techniques; however, the study used a relatively small dataset and excluded runtime system metrics, which remains a key gap this research directly targets.

5.4 Semantic and SQL Text-Based Models

Zolaktaf et al. (2020) used CNN and LSTM models on raw SQL text to predict runtime and other query properties. Character-level CNNs showed strong performance, though large training datasets are required, a constraint addressed by our benchmark-driven data collection approach.

5.5 Advanced Models: Learning-to-Rank and Graph Neural Networks

Zhu et al. (2023) introduced a learning-to-rank approach (Lero), which compares alternative query plans rather than predicting execution time directly, achieving improvements over traditional optimizers. Strausz et al. (2024) applied Graph Neural Networks for multi-engine query routing, though their approach depends on synthetic data and lacks real-world cost considerations.

5.6 Hybrid Machine Learning Approaches

Akdere et al. (2012) demonstrated that machine learning models can effectively predict query performance by learning from historical execution data, improving accuracy over traditional estimation methods.

5.7 Identified Research Gap

Existing studies show progress using query plan features, semantic SQL features, and system-level metrics individually, but rarely in combination (Pathak & Mankodi, 2025). Many also rely on limited or synthetic datasets, reducing their effectiveness in dynamic real-world environments. A clear gap exists for a unified multi-modal model that simultaneously integrates all three feature categories, trained on large real-world benchmark data, to achieve more accurate and generalizable SQL query performance prediction.

6. Research Methodology

This research adopts a quantitative experimental approach using PostgreSQL and standard benchmark datasets (TPC-H and TPC-DS). A dataset of over 5000 query instances will be generated, collecting execution time, query execution plans, and runtime system-level metrics under varied workload conditions.

Multi-modal features will be extracted across three categories: (a) query plan features, including operator types, cardinality estimates, cost values, and tree structure; (b) semantic SQL features, including join count, predicate complexity, and subquery nesting depth; and (c) system-level metrics, including CPU utilization, memory usage, and disk I/O at query execution time. The dataset will be preprocessed, normalized, and subjected to feature selection to maximize model performance.

Machine learning models, including Linear Regression (baseline), Random Forest, and XGBoost, will be trained and evaluated using MAE, MSE, and R^2 metrics. Results will be rigorously compared against PostgreSQL's native cost-based estimates. A lightweight prototype prediction API will be developed to demonstrate practical deployability.

6.1 Justification of Methodology

The quantitative experimental approach enables objective comparison between machine learning models and traditional cost-based estimation methods using measurable, reproducible metrics. TPC-H and TPC-DS benchmark datasets ensure standardization, diversity, and reproducibility of results. Extracting multi-modal features from query plans, SQL semantics, and system-level metrics provides a more comprehensive representation of query behavior compared to single-feature approaches in the literature. Random Forest and XGBoost are selected for their demonstrated ability to capture complex, non-linear relationships and their proven effectiveness in similar prediction tasks (Pathak & Mankodi, 2025; Saritha & Sadanandam, 2024). The use of MAE, MSE, and R^2 ensures a reliable, multidimensional assessment of model performance, supporting the development of an accurate and generalizable prediction model while maintaining scientific validity and practical applicability.

7. Expected Project Deliverables

- Written Research Proposal
- Data collection and feature extraction pipeline
- Labelled dataset of at least 5000 SQL queries with multi-modal features and execution times
- Trained and evaluated Random Forest and XGBoost regression models
- Comparative evaluation report with statistical analysis vs. PostgreSQL cost estimates and the baseline proposed by Pathak and Mankodi (2025)
- Lightweight prediction API accepting SQL queries and returning estimated execution times
- Final research report and presentation

8. Project Schedule

The following Gantt chart presents the planned 12-week project timeline. Colored cells indicate active work periods. Team members are explicitly assigned to each milestone in accordance with the project guidelines.

Phase / Task	Month 1	Month 2	Month 3	Month 4	Month 5	Month 6
Literature Review and the project planning	All members					
Data collection and pre processing		Kirisigan, Nirojan				
Model development		Thivya, Jashvanth	Thivya, Jashvanth	Thivya, Jashvanth		
Case studies, testing, and feedback				Kirisigan, Nirojan	Kirisigan, Nirojan	

Evaluation and final documentation						All members
------------------------------------	--	--	--	--	--	-------------

9. References

- Akdere, M., Cetintemel, U., Riondato, M., Upfal, E., & Zdonik, S. (2012). Learning-based query performance modeling and prediction. *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*.
- Kleerekoper, A., Navaridas, J., & Luján, M. (2019). Can the optimizer cost be used to predict query execution times? *International Workshop on Big Data Management*.
- Lan, H., Bao, Z., & Peng, Y. (2021). A survey on advancing the DBMS query optimizer: Cardinality estimation, cost model, and plan enumeration. *Data Science and Engineering*, 6(1), 86–101.
- Li, J., et al. (2012). Towards accurate and fast query selectivity estimation. *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*.
- Li, J., König, A. C., Narasayya, V., & Chaudhuri, S. (2012). Robust estimation of resource consumption for SQL queries using statistical techniques. *Proceedings of the VLDB Endowment*, 5(11), 1555–1566.
- Marcus, R., & Papaemmanouil, O. (2019). Plan-structured deep neural network models for query performance prediction. *Proceedings of the VLDB Endowment*, 12(11), 1733–1746.
- Pathak, U., & Mankodi, A. (2025). Redefining cost estimation in database systems: The role of execution plan features and machine learning. *arXiv*.
- PostgreSQL Global Development Group. (2024). *EXPLAIN ANALYZE*. <https://www.postgresql.org/docs/current/sql-explain.html>
- Saritha, B., & Sadanandam, M. (2024). Optimizing SQL query execution time: A hybrid approach using machine learning and deep learning techniques. *Journal of Computer Science*, 20(4).
- Strausz, A., Pardon, N., & Giurgiu, I. (2024). A learned cost model-based cross-engine optimizer for SQL workloads. *Proceedings of the VLDB Endowment*.
- Transaction Processing Performance Council. (2023). *TPC-H and TPC-DS benchmark specifications*. <http://www.tpc.org>
- Vasilenko, N., Demin, A., & Ponomaryov, D. (2024). Adaptive cost model for query optimization. *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*.
- Wu, W., Chi, Y., Hacigümüş, H., & Naughton, J. F. (2013). Towards predicting query execution time for concurrent and dynamic database workloads. *Proceedings of the VLDB Endowment*, 6(10), 925–936.

Zhu, R., Chen, W., Ding, B., Chen, X., Pfadler, A., Wu, Z., & Zhou, J. (2023). Lero: A learning-to-rank query optimizer. *Proceedings of the VLDB Endowment*, 16(6), 1466–1479.

Zolaktaf, Z., Milani, M., & Pottinger, R. (2020). Facilitating SQL query composition and analysis. *Proceedings of the ACM SIGMOD International Conference on Management of Data*.